

---

**Assignment 1**  
**Classification and**  
**Prediction**  
**Machine Learning I**

---

**Dual Bachelor in Data Science and Engineering**  
**and Telecommunications Engineering**

---

**Alejandro Barroso - 100499081**

**Juan José Rosales - 100499176**

**Group 196**

---

# Index

|                                           |    |
|-------------------------------------------|----|
| Phase 1: Instances collection.....        | 3  |
| Phase 2:Classification.....               | 5  |
| Phase 3: Building an automatic agent..... | 6  |
| Phase 4: Prediction.....                  | 7  |
| Questions.....                            | 9  |
| Conclusions.....                          | 10 |

## Phase 1: Instances Collection

For this assignment, a complete collection of data instances must be obtained from the game of the snake. We needed to represent the whole game with variables, taking also into account the future phases of the project (classification of the direction variable and prediction) The features we decided that were useful for this assignment were:

| ATTRIBUTE          | TYPE    | VALUES               |
|--------------------|---------|----------------------|
| snake_pos_X        | Numeric |                      |
| snake_pos_Y        | Numeric |                      |
| food_posX          | Numeric |                      |
| food_posY          | Numeric |                      |
| RIGHT              | Nominal | {1,0}                |
| LEFT               | Nominal | {1,0}                |
| UP                 | Nominal | {1,0}                |
| DOWN               | Nominal | {1,0}                |
| good_move          | Nominal | {RIGHT,LEFT,UP,DOWN} |
| previous_direction | Nominal | {1,2,3,4}            |
| Body_up            | Nominal | {1,0}                |
| Body_down          | Nominal | {1,0}                |
| Body_left          | Nominal | {1,0}                |
| Body_right         | Nominal | {1,0}                |
| Body_length        | Numeric |                      |
| Distance_to_food   | Numeric |                      |
| Distance_x         | Numeric |                      |
| Distance_y         | Numeric |                      |
| Rel_pos            | Nominal | {1,2,3,4}            |
| X_rel_pos          | Nominal | {RIGHT,LEFT,SAME}    |
| Y_rel_pos          | Nominal | {UP,DOWN,SAME}       |
| Same_x             | Nominal | {1,0}                |
| Same_y             | Nominal | {1,0}                |
| Score              | Numeric |                      |
| Direction          | Nominal | {1,2,3,4}            |
| Next_score         | Numeric |                      |

Many of the attributes used were variables that already existed from the previous tutorials, such as the snake's head and food positions. We coded the direction in a clockwise sense (UP – 1; RIGHT – 2; DOWN – 3 and LEFT – 4) and so was done for the previous\_direction one. However, all the other attributes were computed using different procedures for each attribute.

- **RIGHT, LEFT, UP, DOWN:** Boolean variables that equal 1 when moving in that position is safe (no collisions) and 0 otherwise.
- **Body\_up, body\_down, body\_right, body\_left:** Boolean variables that equal 1 when there is a body part in that direction from snake's head and 0 otherwise. We computed it by comparing the position of the snake with all the snake's body portions.
- **distance\_to\_food, distance\_x, distance\_y:** Numerical variables that measures the different distances from the head to the food. Both the distance\_x and distance\_y use the Manhattan distance while distance\_to\_food gives the total Euclidean distance.
- **Rel\_pos:** Returns the quadrant (1, 2, 3 or 4) in which the food is with respect to the snake's head.
- **Same\_x, same\_y:** Boolean that is 1 when the food and the head coincide in x or y.

- **X\_rel\_pos, Y\_rel\_pos:** returns the direction in which the food is with respect to the snake's head and equals to "SAME" when they coincide.
- **Good\_move:** this function was computed by combining the information given by the group of variables RIGHT, LEFT, UP, DOWN (to know which directions are safe) with X\_rel\_pos and Y\_rel\_pos . This function always returns a safe direction, but prioritizing the one that brings the snake closer to the food if possible.

Finally, we needed to save also the next\_score variable in each instance. To do so, we divided the instance "type" in 3 to know exactly what to print:

- The first type was the first instance of each game, which prints in a new line all variables except next\_score.
- The second type are the instances obtained from ticks where the snake does not die (direction is safe), printing the score at the end of the previous line and in the current line all variables except next\_score.
- Finally, the instances in which the player dies, the score is printed in the previous line and in the current instance all variables are printed, finishing with the game score again.

In this way, instances that represent the snake dying have the same score for current\_score and next\_score. Some of these features may not be useful for both classification and regression tasks. Therefore, we decided to include all of them in our initial data sets (training and testing) and some preprocessing will be done in the next assignment phases.

## Phase 2: Classification

The aim of this section is to obtain a classification model that decides the direction that the snake should perform, given the state of the game.

The first step is the **data preprocessing**, which must be done in both training and test sets. Since we cannot use attributes related to the future, we deleted the next\_score feature. Besides, since this phase is focused on classifying the direction using only current state attributes, we decided to remove the following features since they are more accurate for a regression task: body\_length, distance\_to\_food, distance\_x, distance\_y and score.

Once we have reduced our data set to have only classification-oriented features, we realized that we had some variables that included the information of the others in a much more efficient way. This was the case of the rel\_pos, same\_x and same\_y, which were optimized with the attributes x\_rel\_pos and y\_rel\_pos; and the safety variables (RIGHT, LEFT, UP, DOWN), which were included in the good\_move feature. Therefore, these attributes have also been removed from our data sets.

Finally, due to the complexity of the variables body\_up, body\_right, body\_down and body\_left; we decided to delete them as well. The resulting data set included:

| ATTRIBUTE          | TYPE    | VALUES               |
|--------------------|---------|----------------------|
| snake_pos_X        | Numeric |                      |
| snake_pos_Y        | Numeric |                      |
| food_posX          | Numeric |                      |
| food_posY          | Numeric |                      |
| good_move          | Nominal | {RIGHT,LEFT,UP,DOWN} |
| Previous_direction | Nominal | {1,2,3,4}            |
| X_rel_pos          | Nominal | {RIGHT,LEFT,SAME}    |
| Y_rel_pos          | Nominal | {UP,DOWN,SAME}       |
| Direction          | Nominal | {1,2,3,4}            |

The next step was selecting the **instance subset** that would achieve the best outcome possible. To do so we created the following data sets to be compared afterwards (all of them include the direction, since is the class variable):

- **Data\_train\_simple**: data set with all the previous features.
- **Data\_train\_norm\_disc**: same variables than the previous one but we decided to transform the numeric attributes into nominal ones. To do so, we first tried discretizing directly the data set. However, this resulted in different bins in training and test set, which may be a problem in the future. Therefore, the solution we came up with was to first normalize the data frame to have all the attributes in the same scale and then discretize them. In this way, we obtained the same bin distribution in train and test sets.
- **Data\_train\_relpos\_gm**: data set with only 3 variables: good\_move, x\_rel\_pos and y\_rel\_pos. We removed the position variables of the snake and the food since we understood that their utility may be included in the relative\_positions variables. The same happened with previous\_direction, which is used to avoid dying due to turning around, since the good\_move variable also detects the possible collision in that direction.
- **Data\_train\_prev\_relpos**: data set with just the x\_rel\_pos and y\_rel\_pos and the previous\_direction. The relative positions are used to get closer to the food while the previous direction is used in order to avoid the snake going backwards and dying.
- **Data\_train\_gm**: data set with just the good\_move attribute. Theoretically, this variable may be enough to obtain a good classification, since it detects collisions while focusing on getting closer to the food.

Now we need to use the different training sets to obtain several classifiers and decide which one is the best. To do so, we are going to use the **weka experimenter** to generate classifiers using the following algorithms:

- **ZeroR** (the benchmark) – assigns the majority class value to all instances.
- **PART** - Builds a partial C4.5 decision tree in each iteration and makes the "best" leaf into a rule.
- **J48** – generates a pruned or unpruned C4.5 decision tree.
- **IBk (k = 1, 3, 5)** - K-nearest neighbors classifier, we decided to use 3 different values for k (neighbors).
- **RandomForest** – generates many decision trees to create a forest to obtain the final outcome.
- **NaiveBayes** - calculates the posterior probability for each class and makes a prediction for the class with the highest probability.
- **SMO** – is an implementation of Support Vector Machines (SVM). This implementation transforms nominal attributes into binary ones. It also normalizes all attributes by default.

| Algorithm\Data      | Simple | Norm-Disc | Relpos_gm | Prev_Relpos | gm    |
|---------------------|--------|-----------|-----------|-------------|-------|
| <b>ZeroR</b>        | 26.99  | 26.99     | 26.99     | 26.99       | 26.99 |
| <b>PART</b>         | 97.32  | 93.77     | 64.10     | 55.66       | 49.34 |
| <b>J48</b>          | 97.48  | 93.52     | 64.11     | 55.66       | 49.34 |
| <b>IB1</b>          | 97.61  | 93.46     | 64.12     | 55.66       | 49.34 |
| <b>IB3</b>          | 97.12  | 91.92     | 64.12     | 55.66       | 49.34 |
| <b>IB5</b>          | 96.71  | 90.63     | 64.05     | 55.66       | 49.34 |
| <b>RandomForest</b> | 97.85  | 93.93     | 64.13     | 55.66       | 49.34 |
| <b>NaiveBayes</b>   | 83.21  | 83.20     | 49.18     | 55.66       | 49.34 |
| <b>SMO</b>          | 91.22  | 90.92     | 64.07     | 55.66       | 49.34 |

---

In view of this outcome, we can see how the accuracy gets lower with a lower number of variables, which makes sense. However, since we do not want the automatic agent to do exactly the same moves we do, we should select one of the datasets that gave a lower accuracy. Therefore, we discard the `Data_train_simple` and the `Data_train_norm_disc`.

On the other hand, the algorithms obtained very similar outcomes for the same data, which gives us the freedom to select any of them to create a model. We decided to use PART and the J48 since they present their results in a visual way, with rules or a tree.

Finally, we trained 6 classifiers (one PART and one J48 for each data set), using the test sets as test option for Weka and obtaining for all of them similar accuracies, around 50%. By analyzing their trees and rules, we decided that they all should work fine for phase 3.

## Phase 3: Building an Automatic Agent

We first tried to prepare the `java_bridge` for this phase in Windows, but we were unable to accomplish it. Therefore, the best solution was to use the ubuntu virtual machine available. There, we downloaded all the required packages and prepared a pycharm project with the training sets, their models, the game script and wekal.

Then, we had to modify the `move_tutorial1(game)` method of the game script. To do so, we created the variables that were used in our data sets and introduced them in a list called `x` that will be used to obtain the final prediction of the direction for each tick.

Finally, when we tried the 6 different models with their corresponding training set (changing for each of them the input attributes in the list `x` that were going to be used) we discovered that all of them accomplished eating some apples. However, we finally decided to select the j48 model trained with the `Data_train_goodmove`, since its capacity to avoid collisions while approximating to the food allowed the snake to live longer and achieve higher scores. The other data sets were discarded even though they managed to eat some apples too.

Analyzing the behavior of the agent, it basically moves first in the horizontal axis until the food is in the same `x` and then moves up or down until it eats the food. However, it does not avoid colliding with its own body. This is the main difference between this smart agent and the one created in the tutorial 1, which was able to avoid any kind of collision except when it was trapped and no direction was safe. On the other hand, the difference with the keyboard one is that we play without following rules being the moves more “random” than the agent’s ones.

## Phase 4: Prediction

### Data preprocessing:

We start from two datasets, one of the games played using keyboard and another that was created using the smart agent. Attributes of both datasets are structured as said before:

#### 1. Relevant Attributes Selection:

| ATTRIBUTE          | TYPE    | VALUES               |
|--------------------|---------|----------------------|
| snake_pos_X        | Numeric |                      |
| snake_pos_Y        | Numeric |                      |
| food_posX          | Numeric |                      |
| food_posY          | Numeric |                      |
| RIGHT              | Nominal | {1,0}                |
| LEFT               | Nominal | {1,0}                |
| UP                 | Nominal | {1,0}                |
| DOWN               | Nominal | {1,0}                |
| good_move          | Nominal | {RIGHT,LEFT,UP,DOWN} |
| previous_direction | Nominal | {1,2,3,4}            |
| Body_up            | Nominal | {1,0}                |
| Body_down          | Nominal | {1,0}                |
| Body_left          | Nominal | {1,0}                |
| Body_right         | Nominal | {1,0}                |
| Body_length        | Numeric |                      |
| Distance_to_food   | Numeric |                      |
| Distance_x         | Numeric |                      |
| Distance_y         | Numeric |                      |
| Rel_pos            | Nominal | {1,2,3,4}            |
| X_rel_pos          | Nominal | {RIGHT,LEFT,SAME}    |
| Y_rel_pos          | Nominal | {UP,DOWN,SAME}       |
| Same_x             | Nominal | {1,0}                |
| Same_y             | Nominal | {1,0}                |
| Score              | Numeric |                      |
| Direction          | Nominal | {1,2,3,4}            |
| Next_score         | Numeric |                      |

We delete the ones related to predict direction (purple) and keep the ones related to next\_score(blue).

The most significant variables are:

**Body\_length:** Length of the snake, positively correlated with the score.

**Distance\_to\_food:** Inversely correlated with the next score, as each move without consuming food reduces the score.

**Distance\_x(y):** similar to distance\_to\_food but within x and y axis. They provide more precise information about how cells away is the food.

#### 2. Outlier Handling:

Identified outliers in the 'next\_score' attribute, such as the imposible value of 15521553 points, likely due to a game iteration issue.

Applied the "RemoveWithValues" filter to eliminate 'next\_score' values exceeding 10000 points, ensuring data integrity and model accuracy.

#### 3. Attribute Transformation:

Transforme categorical attributes, such as 'same\_x' and 'same\_y', to binary format using the "NominalToBinary" filter. Each class is represented by two dummy variables, simplifying the input for regression algorithms.

#### 4. Normalization:

Use the "Normalize" filter to scale all attribute values between 0 and 1. This normalization process enhances model performance by deleting potential biases arising from high measurement units.

After this data preprocessing process we have 4 models:

- **Preprocessed\_keyboard**: dataset created with keyboard with the most relevant variables.
- **Preprocessed\_agent**. dataset created by the agent with the most relevant variables.

We perform the experiment with all of them to then be able to compare and take insights.

Now we need to use the different training sets to obtain regression models and decide which one is the best. To do so, we are going to use the **weka experimenter** to generate regression models using the following algorithms:

- **Linear Regression**: This model fits a linear relationship between input features and output.
- **Multilayer Perceptron**: A type of neural network model with multiple layers of nodes.
- **Simple Linear Regression**: This model assumes a linear relationship between the input feature and output. It's a basic regression technique to model the relationship between one independent variable and one dependent variable.
- **SMO (Sequential Minimal Optimization)**: SMOrg is an efficient algorithm for training support vector machines (SVMs).

#### RESULTS:

**Measurments are: root mean squared error ( standard deviation): the lowest the closest to the real value.**

| Dataset/algorithm | LinearRegression | MultilayerPerceptron | SimpleLR     | SMOrg        |
|-------------------|------------------|----------------------|--------------|--------------|
| <b>Keyboard</b>   | 7.48(5.33)       | 8.42(4.98)           | 7.49(5.33)   | 7.44(5.38)   |
| <b>Agent</b>      | 4.93(4.41)       | 5.29(4.18)           | 4.97(4.38) v | 4.95(4.40) v |

##### 1. Linear Regression:

Keyboard: 7.48(5.33)

Agent: 4.93(4.41)

Analysis: Linear Regression performs moderately, with higher errors compared to other algorithms for both the Keyboard and Agent datasets.

##### 2. Multilayer Perceptron:

Keyboard: 8.42(4.98)

Agent: 5.29(4.18)

Analysis: Multilayer Perceptron demonstrates slightly worse performance than Linear Regression for the Keyboard dataset. However, it shows improvement for the Agent dataset, indicating better adaptability to different data distributions.

##### 3. Simple LR

Keyboard: 7.49(5.33)

Agent: 4.97(4.38) v

Analysis: Simple Linear Regression performs similarly to Linear Regression, showing moderate performance for the Keyboard dataset. However, it shows a better performance for the Agent dataset, suggesting an improvement for simpler models.

---

#### 4. **SMOrg:**

Keyboard: 7.43(5.38)

Agent: 4.95(4.40) v

Analysis: SMOrg shows a similar performance to Linear Regression and Simple Linear Regression for the Keyboard dataset. However, it shows slightly better performance for the Agent dataset, indicating the possibility of capturing nonlinear relationships.

Overall, all models performed similarly. We can highlight a worse relative performance of Multilayer Perceptron. It may be due to its complexity. Simple LR and LR perform very similar, which makes sense since they are very similar models. SMOrg is really close to both linear regression models, performing slightly better in the agent dataset. SMOrg main difference from linear regression it's the possibility of capturing nonlinear relations. This capability seems to describe the slight improvement in performance. Our selected model would be SMOrg due to its good performance and capability of capturing a wide range of relations.

---

## Questions

1. **What is the difference between learning these models with instances coming from a human-controlled agent and an automatic one?**

Rules followed by humans are completely random so when creating this model with human-controlled agents there is no possible way to predict with full accuracy the behavior since it is random.

On the other hand, a model created using data from a smart agent may be easier to predict since all movements done by the agent are taken following some predefined rules.

We can prove this idea by looking at the results of phase 4 in which regression models perform better when using the agent dataset.

2. **If you wanted to transform the regression task into classification, what would you have to do? What do you think could be the practical application of predicting the score?**

For the purpose of predicting next\_score making use of a classification model instead of a regression model nominal variables are needed. To achieve this, we can discretize all numerical variables into intervals that can be interpreted by classification models. We would need to perform this transformation also in next\_score so we would just be able to predict next\_score between intervals(classification).

This gives the possibility of computing the percentage of correctly classified next\_scores intervals. It could help us to maximize score once our model is good enough but does not reach high scores. With this classification we can see relations between directions and scores and choose the best direction in order to get the maximum score. This can be interpreted as choosing the shortest path to the food.

3. **What are the advantages of predicting the score over classifying the action? Justify your answer.**

The main advantage of predicting the score over classifying the action is that it maximizes the score of the game, which is the main objective of playing it. Just classifying the action may help eat the food, but the process to do so can make the score lower if the path followed is not the shortest one.

4. **Do you think that some improvement in the ranking could be achieved by incorporating an attribute that would indicate whether the score at the current time has dropped?**

I think it can improve the process of selecting the next direction of the game. Adding this variable would introduce the possibility of selecting the direction that maximizes the score. We think it would be a perfect approach if a good model is already created but it does silly moves or it does not reach high scores.

---

## Conclusions

During this assignment we encountered several technical issues, most of them regarding phase 3 of the project. A lot of time was spent trying to make the `java.bridge` work in windows. However, our efforts were useless since in the end we had to use the Virtual Machine. Besides, a lot of errors were constantly appearing when trying to run the new agent, most of them having an unrecognizable name. The process of getting rid of these errors was a very tedious and exhausting process, but we managed to make it work at the end.

Apart from these technical issues, we also faced several problems that forced us to start from scratch. These problems were all related to the usefulness of the first data set that we created, since we overestimated the capacity of classification of the models. Therefore, we had to go back and create more attributes that represented each game state in a more specific and useful way.

However, the general assignment made us realize of other possible fields in which machine learning can be applied. The most obvious one due to the relationship of this project with gaming is the creation of NPC's for videogames, where they should know what to do at each tick of the game. Besides, it may be also useful in the biology research field, since given some properties a classification can be made; and for the economics field, being the regression task very useful to predict gains or losses.

In conclusion, this project has showed us the whole process of using and applying machine learning on a real task. Besides, being able to see how the final agent of phase 2 worked fine after our efforts, made us feel satisfied with our work. Weka is a useful tool to implement machine learning, but finding documentation to make a good use of the program is really hard. There is not much information online and processes like using the experimenter are not easy. Therefore, these lack of extra-documentation joined to the technical issues encountered with `java.bridge` made it hard to enjoy the whole assignment process..